

TABLE OF CONTENTS

S.No	Date	Name of the Experiment	Page No	Marks	Signature
1	16-8-24	Write programs in Python Language to demonstrate the working of following constructs with possible test cases: a) do...while b) while...do c) if ...else d) switch e) for			
2	23-8-24	Write a program in Python language for Matrix multiplication fails. Introspect the causes for its failure and write down the possible reasons for its failure.			
3	30-8-24	Prime number or Not			
4	6-9-24	Armstrong Number			
5	13-9-24	Binary search			
6	20-9-24	Palindrome			
7	27-9-24	Sorting			
8	4-10-24	Take ATM system and study its system specifications and report various bugs			
9	11-10-24	Test cases for banking applications			
10	18-10-24	Test plan document for library application			
11	25-10-24	Pytest Installation			
12	25-10-24	Pytest Python program for Addition			
13	1-11-24	Pytest Python program for Sum of Digits			
14	8-11-24	Pytest Python program for Fibonacci Series			
15	15-11-24	Study of Selenium Web Testing Tool			

Ex.No: 1 Write programs in Python Language to demonstrate the working of following constructs with possible test cases: a) do...while b) while...do c) if ...else d) switch e) for

DATE:

REGISTER NUMBER : 212222040036

AIM:

To write python programs for do...while, while, for, switch and if...else and test with possible test Cases

Algorithm:

1. Start the program.
2. Create separate files for each given program.
3. Write simple program for each construct.
4. the program with possible test cases.
5. Stop the program.

Program:

i.)do...while:

```
def display():
    start=input("Enter a positive value for START: ")
    end=input("Enter a positive value for END: ")
    if start.isnumeric() and end.isnumeric():
        while True:
            start=int(start)
            end=int(end)
            print(start,end=' ')
            if start<end:
                start+=1
            else:
                break
    else:
        print("Enter a valid positive number.")
    display()
```

ii.) while...do

```
start=input("Enter a positive value for START: ")
end=input("Enter a positive value for END: ")
if start.isnumeric() and end.isnumeric():
    start=int(start)
    end=int(end)
    while start<end:
        print(start)
        start+=1
else:
    print("Enter a valid positive number.")
```

iii.) switch

```
def switch():
    switcher={
        0:"even",
        1:"odd"
    }
    n=input('Enter a value for N: ') try:
        n=int(n)
        print(switcher[n%2])
    except ValueError:
        print("Enter a valid number.")
    switch()
```

iv.) if else

```
def compare():
    a=input("Enter a value for A: ")
    b=input("Enter a value for B: ")
    try:
        a=int(a)
        b=int(b)
        if a>b:
            print("A is greater than")
        elif a<b:
            print("B is greater than")
        else:
            print("A is equal to B")
    except ValueError:
        print("Enter a valid number.")
```

v.) for

```
def iterate():
    string=input("Enter a string: ")
    for i in string:
        print(ord(i),end=" ")
    iterate()
```

Output:

i.)do...while:

```
Positive numbers
Enter a positive value for START: 1
Enter a positive value for END: 4
1 2 3 4

Negative numbers
Enter a positive value for START: -10
Enter a positive value for END: 5 Enter
a valid positive number.

Character and string input
Enter a positive value for START: hello
Enter a positive value for END: y
Enter a valid positive number.

Null input
Enter a positive value for START:
Enter a positive value for END:
Enter a valid positive number.
```

ii.) while...do

```
Positive numbers
Enter a positive value for START: 1
Enter a positive value for END: 4
1 2 3 4

Negative numbers
Enter a positive value for START: -10
Enter a positive value for END: 5 Enter
a valid positive number.

Character and string input
Enter a positive value for START: abc
Enter a positive value for END: 100
Enter a valid positive number.

Null input
Enter a positive value for START:
Enter a positive value for END:
Enter a valid positive number.
```

iii.) switch

```
Positive numbers
Enter a value for N: 1
odd

Negative numbers
Enter a value for N: -10
even

Character and string input
Enter a value for N: hello
Enter a valid number.

Null input
Enter a value for N:
Enter a valid number.
```

iv.) if else

```
Positive numbers
Enter a value for A: 1
Enter a value for B: 1
A is equal to B.
```

```
Negative numbers
Enter a value for A: -10
Enter a value for B: 5 B
is greater than A.
```

```
Character and string input
Enter a value for A: hello
Enter a value for B: y
Enter a valid number.
```

```
Null input
Enter a value for A:
Enter a value for B:
Enter a valid number.
```

v.) for

```
Characters
Enter a string: say
115 97 121
```

```
Number
Enter a string: 1543
49 53 52 51
```

```
Null input
Enter a string:
```

Result:

Thus, the python program to demonstrate the working of given constructs is implemented and the output is verified successfully.

Ex.No: 2 Matrix Multiplication

DATE:

REGISTER NUMBER : 212222040036

AIM:

Write a python program for matrix multiplication and inspect for failures.

Algorithm:

Algorithm:

1. Start the program.
2. Create empty list formatrix1, matrix2 and result.
3. Get the rows and columns count from the user.
4. Get the values of two matrix.
5. Perform matrix multiplication and store the answer in result.
6. Stop the program.

Program:

```
r1, c1 = input("Enter row and column count in matrix 1: ").split()
r2, c2 = input("Enter row and column count in matrix 2: ").split()

matrix1 = []
matrix2 = []
result = []

# Check if input values are numeric
if r1.isnumeric() and c1.isnumeric() and r2.isnumeric() and c2.isnumeric():
    r1 = int(r1)
    r2 = int(r2)
    c1 = int(c1)
    c2 = int(c2)

    # Check if matrix multiplication is possible
    if c1 != r2:
        print("Matrix multiplication not possible")
    elif max(r1, c1, r2, c2) > 20 or min(r1, c1, r2, c2) == 0:
        print("Matrix multiplication not possible")
    else:
        # Input matrix 1

        print("Enter matrix 1:")
        for i in range(r1):
            a = []
            for j in range(c1):
                a.append(int(input(f"Element [{i+1}][{j+1}] for matrix 1: ")))
            matrix1.append(a)

        # Input matrix 2
        print("Enter matrix 2:")
        for i in range(r2):
            a = []
            for j in range(c2):
                a.append(int(input(f"Element [{i+1}][{j+1}] for matrix 2: ")))
            matrix2.append(a)

        # Matrix multiplication logic
        for i in range(r1):
            inter = []
            for j in range(c2):
                sum = 0
                for k in range(c1): # Should be c1 because it matches r2
                    sum += matrix1[i][k] * matrix2[k][j]
                inter.append(sum)
            result.append(inter)

        # Output the result
        print("Resultant matrix after multiplication:")
        for i in range(r1):
            for j in range(c2):
                print(result[i][j], end=" ")
            print()

    else:
        print("Enter valid numeric values for matrix dimensions.")
```


Output:

```
Enter the size of a: 2 3
Enter the size of b: 2 3
Matrix multiplication is not possible.
Reason to fail: to do multiplication of matrices the number of columns in matrix -a[]
should be
equal to the number of rows in matrix in b[]
```

```
Enter the size of a: p q
Enter the size of b: q s
Matrix multiplication is not possible.
Reason to fail: to do multiplication of matrices the number of columns in matrix -a[]
should be equal to number of rows in matrix -b[], and rows & columns should be integer
values.
```

```
Enter the size of a: 1.5 2
Enter the size of b: 2 3
Matrix multiplication is not possible.
Reason to fail: to do multiplication of matrices the number of columns in matrix -a[]
should be
equal to number of rows in matrix -b[], and rows & columns should be integer values.
```

```
Enter the size of a: 350
480 Enter the size of b:
480 620
Matrix multiplication is not possible.
Reason to fail: size of buffer will be not be sufficient to handle this
multiplication.
```

```
Enter the size of a: -
1 -2 Enter the size of
b: -2 3
Matrix multiplication is not possible.
Reason to fail: to do multiplication of matrices the number of columns in matrix -a[]
should
be equal to number of rows in matrix -b[], and rows & columns should be positive
integer values.
```

Result:

Thus, the python program for matrix multiplication is implemented and the causes for its failure is introspected successfully.

Ex.No: 3 To check the number is prime or not and inspect for failures.

DATE:

REGISTER NUMBER : 212222040036

AIM:

Write a python program to check the number is prime or not and inspect for failures.

Algorithm:

1. Start the program.
2. Get the number to be checked from the user.
3. If the number is less than or equal to 1, return "Not Prime".
4. If the number is 2, return "Prime".
5. Start the iteration from 3, For each iteration:
6. If the number is divisible by the current iteration value, return "Not Prime".
7. If the number is not divisible by any value from 2 to the square root, return "Prime".
8. Stop the program.

Program:

```
num = input()
flag = 0

if num.isnumeric():
    z = int(num)
    if z == 2:
        flag = 1
    if z > 2:
        for i in range(2, z // 2):
            if z % i == 0:
                flag = 0
                break
        else:
            flag = 1
    if flag == 1:
        print("Prime Number")
    else:
        print("Not a Prime Number")
else:
    print("Enter a Positive Number")
```

Output:

```
Enter a Positive number: 2
Prime number

Enter a Positive number: 5
Prime number

Enter a Positive number: krishna
Enter a positive number
Reason: Strings are not allowed

Enter a Positive number: -1
Enter a positive number
Reason: Negative number are not allowed.

Enter a Positive number: 1
Not a Prime number
Reason: The entered number is not a prime number.
```

Result:

Thus, the python program to check the number is prime or not is implemented and the output is verified successfully.

Ex.No: 4 check the given number is Armstrong number or not and inspect for failures.

DATE:

REGISTER NUMBER : 212222040036

AIM:

Write a python program to check the number is Armstrong number or not and inspect for failures.

Algorithm:

1. Start the program.
2. Read an integer input number.
3. Initialize the variables current_digit, sum = 0, and num = number.
4. Repeat Steps 5 to 7 until num > 0
5. current_digit = (num % 10).
6. sum = sum + (current_digit * current_digit * current_digit). 7. Stop the program.
7. num = num / 10.
8. Check if sum == number. If true, print "It is an Armstrong Number." Otherwise, print "It is not an Armstrong Number."
9. Stop the program.

Program:

```
x = input("Enter the input: ")
if x.isnumeric():
    x = int(x)
    temp = x
    cube = 0
    while temp > 0:
        digit = temp % 10
        cube = cube + (digit ** 3)
        temp //= 10
    if cube == x:
        print("Armstrong Number")
    else:
        print("Not Armstrong Number")
else:
    print("Enter a Positive Integer.")
```

Output:

```
Enter the Input: 0
Armstrong Number

Enter the Input: 153
Armstrong Number

Enter the Input: 44
Not a Armstong Number

Enter the Input: -153
Not a Armstrong Number
Reason: The input can only be a positive integer.

Enter the Input: Teja
Enter a positive integer
Reason: The input can only be of a positive integer.

Enter the Input: !@#
Enter a positive integer
Reason: The special characters are not allowed.
```

Result:

Thus, the python program to check the number is Armstrong number or not implemented and the output is verified successfully.

Ex.No 5 Binary Search

Date :

Register Number:212222040036

Aim

Write a program in Python language to search a given element is present in the list using Binary search. Introspect the causes for its failure and write down the possible reasons for its failure.

Algorithm:

1. Start the program.
2. Get the list from the user
3. Get the element to be searched
4. Compare the mid element with the key, if same return the index
5. If key is greater, search it in the right side, else search it in the left side.
6. If not found return -1
7. Stop the program.

Program:

```
def binary_search(arr, x):
    low = 0
    high = len(arr) - 1
    mid = 0

    while low <= high:
        mid = (high + low) // 2

        if arr[mid] < x:
            low = mid + 1
        elif arr[mid] > x:
            high = mid - 1
        else:
            return mid
    return -1

arr = [2, 3, 4, 10, 40]
x = input("Enter the element to be searched: ")
try:
    x = int(x)
    result = binary_search(arr, x)

    if result != -1:
        print("Element is present at index",
str(result))
    else:
        print("Element is not present in array")
except ValueError:
    print("Enter a valid input!")
```

Output:

```
Enter the element to be searched: 5
Element is not present in array
```

```
Enter the element to be searched: 4
Element is present at index 2
```

```
Enter the element to be searched: h
Enter a valid input!
```

```
Enter the element to be searched:
Enter a valid input!
```

```
Enter the element to be searched: -8
Element is not present in array
```

```
Enter the element to be searched: 88
Element is not present in array
```

```
Enter the element to be searched: jk
Enter a valid input!
```

```
Enter the element to be searched: 789
Element is not present in array
```

```
Enter the element to be searched: 40
Element is present at index 4
```

Result:

Thus, the python program of binary search is implemented and the output is verified successfully.

Ex.No: 6 To check whether the string is Palindrome and generate test cases.

DATE: 4/10/24

name : s dharshan

REGISTER NUMBER : 212221040036

AIM:

Write a Python program to check whether the string is Palindrome and generate test cases.

Algorithm:

1. Start
2. Get an input from the user by prompting
3. Run a loop from 0 to len/2.
4. Check if the characters are the same both from the start and the end till len/2.
5. If it is, return the result that it is a palindrome.
6. Else, return that it is not a palindrome.
7. Stop the program.

Program:

```

def Palindrome(string):
    for i in range(0, int(len(string)/2)):
        if string[i] != string[len(string)-i-1]:
            return False
    return True

s = input("Enter a string: ")
c = 1

for i in s:
    if not i.isalpha():
        c = 0
        break

if c == 0:
    print("Enter a valid string")
else:
    answer = Palindrome(s)
    if answer:
        print("The given string is a palindrome")
    else:
        print("The given string is not a palindrome")

```

Output:

The screenshot displays a Python IDE with two panes. The left pane shows the command prompt output for the program, and the right pane shows the source code.

Command Prompt Output (Left Pane):

```

===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: abcd
The given string is not a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: level
The given string is a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: level
The given string is a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: madam
The given string is a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: racecar
The given string is a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: 121
Enter a valid string
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: 801018
Enter a valid string
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: k%
Enter a valid string
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string:
The given string is a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: Hello
The given string is not a palindrome
>>>
===== RESTART: C:/Users/SEC/AppData/Local/Programs/Python/Python311/STL66.py =====
Enter the string: $123
Enter a valid string
>>>

```

Source Code (Right Pane):

```

def Palindrome(string):
    for i in range(0, int(len(string) / 2)):
        if string[i] != string[len(string) - i - 1]:
            return False
    return True

s = input("Enter the string: ")

is_valid = True
for i in s:
    if not i.isalpha():
        is_valid = False
        break

if not is_valid:
    print("Enter a valid string")
else:
    if Palindrome(s):
        print("The given string is a palindrome")
    else:
        print("The given string is not a palindrome")

```

Result:

Thus, a program to check palindrome has been written and test cases have been written and verified successfully.

Ex.No: 7 Sorting.

DATE: 24/09/2024

REGISTER NUMBER :212222040036

AIM:

Write a python program for sorting and inspect for failures.

Algorithm:

1. Start
2. Get an input from the user by prompting
3. Run a loop form 0 to len/2.
4. Check if the characters are the same both from the start and the end till len/2.
5. If it is, return the result that it is a palindrome.
6. Else, return that it is not a palindrome.
7. Stop the program.

Program:

```
n=int(input("Enter the number of elements:"))
arr=[]
try:
    for i in range(n):
        a=float(input("Enter the element:"))
        arr.append(a)
    for i in range(n):
        for j in range(n):
            if(arr[i]<arr[j]):
                temp = arr[i]
                arr[i] = arr[j]
                arr[j] = temp
    print("The array after sorting: ")
    for i in range(n):
        print(arr[i],end=' ')
except ValueError:
    print("Enter a valid number")
```

Output:



```
Enter the number of elements: 15
Enter element 1: 45
Enter element 2: 45
Enter element 3: 54
Enter element 4: 87
Enter element 5: 54
Enter element 6: 54
Enter element 7: 548
Enter element 8: 545
Enter element 9:
Please enter a valid number.

=== Code Execution Successful ===
```

Result:

Thus, a program to check palindrome has been written and test cases have been written and verified successfully.

Ex.No: 8 ATM Applicationn

DATE:01.10.2024

REGISTER NUMBER :212222040036

AIM:

For ATM system study its system specifications and report various bugs

Purpose:

This document describes the software requirements and specification (SRS) for an automat teller machine (ATM) network. The document is intended for the customer and the develope (designers, testers, maintainers). The reader is assumed to have basic knowledge of bank accounts and account services. Knowledge and understanding of Unified Modeling Language (UML) diagrams is also required.



Scope:

The software supports a computerized banking network called _Bank24,,. The network enables customers to complete simple bank account services via automated teller machines (ATMs) that may be located off premise and that need not be owned and operated by the customer's bank. The ATM identifies a customer by a cash card and password. It collects information about a simple account transaction (e.g., deposit, withdrawal, transfer, bil payment), communicates the transaction information to the customer's bank, and dispenses cash to the customer. The banks provide their own software for their own computers. The _Bank24,, software requires appropriate record keeping and security provisions. The Software must handle concurrent accesses to the same account correctly.



Intended Audience:

The intended audience of this SRS consists of:

Software designers



☐

☐

☐

☐

☐

Systems engineers

/Software developers

Software testers

Customers

The actors of the system are:

User

1.

2.

3.

ATM Machine

Bank

Product Perspective:

An automated teller machine (ATM) is a computerized telecommunications device that provides the customers of a financial institution with access to financial transactions public space without the need for a human clerk or bank teller. On most modern ATMs, the customer is identified by inserting a plastic ATM card with a magnetic stripe or a plast smartcard with a chip, that contains a unique card number and some security information,



such as an expiration date or CVC (CVV). Security is provided by the customer entering a personal identification number (PIN).

Product Functions:

Using an ATM, customers can access their bank accounts in order to make cash withdrawals (or credit

card cash advances) and check their account balances. The functions of the system are:

1. Login
2. Get Balance Information
3. Withdraw Cash

Operative Environments:

The hardware, software and technology used should have following specifications:

- ☐ Ability to read the ATM card.
- ☐ Ability to count the currency notes.
- ☐ Touch screen for convenience.
- ☐ Keypad(in case touchpad fails)
- ☐ Continuous power supply.
- ☐ Ability to connect to bank's network.
- ☐ Ability to validate user.

Design/implementation constraints:

Login:

Validate Bank Card

- ☐ Validate for Card Expiration Date
- ☐ Validate that the card's expiration date is later than today's date
- ☐ If card is expired, prompt error message "Card is expired"

Validate for Stolen or Lost Card

- ☐ Validate that the card is not reported lost or stolen
- ☐ If card is lost, prompt error message, "Card has been reported lost"
- ☐ If card is stolen, prompt error message, "Card has been reported stolen"

Validate for Disabled Card

- ☐ Validate that the card is not disabled
- ☐ If card is disabled, prompt error message, "Card has been disabled as of <expiration date>"

Validate for Locked Account Validate

that the account is not locked

- ☐ If account is locked, prompt error message "Account is locked"

Validate PIN

- ☐ Validate that the password is not blank
- ☐ If PIN is blank, prompt error message "Please provide PIN"
- ☐ Validate that the password entered matches the password on file
- ☐ If password does not match, prompt error message "Password is Incorrect"

Lock Account

- ☐ If number of consecutive unsuccessful logins exceeds three attempts, lock account.

Maintain Consecutive Unsuccessful Login Counter

Increment Login Counter

For every consecutive Login attempt, increment logic counter by 1.

Reset login counter to 0 after login is successful.

Get Balance Information

Withdraw Cash

Transfer Funds

Assumptions and Dependencies:

- ☐ Hardware never fails
- ☐ ATM casing is impenetrable
- ☐ Limited number of transactions per day (sufficient paper for receipts)
- ☐ Limited amount of money withdrawn per day (sufficient money)

External Interface Requirements

User interfaces

The customer user interface should be intuitive, such that 99.9% of all new ATM users are able to complete their banking transactions without any assistance.

Hardware interfaces

The hardware should have following specifications:

- ☐ Ability to read the ATM card
- ☐ Ability to count the currency notes
- ☐ Touch screen for convenience
- ☐ Keypad (in case touchpad fails)
- ☐ Continuous power supply
- ☐ Ability to connect to bank's network
- ☐ Ability to take input from user
- ☐ Ability to validate user

Software interfaces

The software interfaces are specific to the target banking software systems. At present, banking systems will participate in the ATM network.

- ☐ State Bank
- ☐ Indian Overseas Bank

Safety requirements:

Must be safe kept in physical aspects, say in a cabin

- ☐ Must be bolted to floor to prevent any kind of theft
- ☐ Must have an emergency phone outside the cabin
- ☐ There must be an emergency phone just outside the cabin
- ☐ The cabin door must have an ATM card swipe slot
- ☐ The cabin door will always be locked, which will open only when user swipes his/her ATM card in the slot & is validated as genuine

Security requirements:

- ☐ Users accessibility is ensured in all the ways
- ☐ Users are advised to change their PIN on first use
- ☐ Users are advised not to tell their PIN to anyone
- ☐ The maximum number of attempts to enter PIN will be three

Possible Bugs:

1. Successful insertion of ATM card
2. Unsuccessful operation due to insert card in wrong angle
3. Unsuccessful operation due to invalid account Ex: other bank card or time expired card
4. successful entry of PIN number
5. unsuccessful operation due to enter wrong PIN number 3times
6. successful selection of language

7. successful selection of account type
8. unsuccessful operation due to invalid account type
9. successful selection of withdraw operation
10. successful selection of amount to be withdrawal
11. successful withdraw operation
12. unsuccessful withdraw operation due to wrong denominations
13. unsuccessful withdraw operation due to amount is greater than day limit
14. unsuccessful withdraw operation due to lack of money in ATM
15. unsuccessful withdraw operation due to amount is greater than possible balance
16. unsuccessful withdraw operation due to transactions is greater than day limit
17. unsuccessful withdraw operation due to click cancel after insert card
18. unsuccessful withdraw operation due to click cancel after insert card & pin number
19. unsuccessful withdraw operation due to click cancel after insert card, pin number &
20. unsuccessful withdraw operation due to click cancel after insert card, pin number, 1 & account type
21. unsuccessful withdrawal operation due to click cancel after insert card, pin number, account type & withdrawal operation
22. unsuccessful withdrawal operation due to click cancel after insert card, pin number, account type, withdrawal operation & amount to be withdraw

Result:

Thus, the ATM system specifications and reporting the various bugs is implemented and output was verified successfully

Ex.No: 9 Test cases for Banking Application

DATE: 08.10.2024

REGISTER NUMBER:212222040036

AIM:

For Banking Applicationsystem study its system specifications and generate test cases.

The characteristics of a Banking application are as follows:

- Multi-tier functionality to support thousands of concurrent user sessions
- Large scale Integration, typically a banking application integrates with numerous other applications such as Bill Pay utility and Trading accounts
- Complex Business workflows
- Real Time and Batch processing
- High rate of Transactions per seconds
- Secure Transactions
- Robust Reporting section to keep track of day-to-day transactions
- Strong Auditing to troubleshoot customer issues
- Massive storage system
- Disaster Management.



The above listed ten points are the most important characteristics of a Banking application.

Banking applications have multiple tiers involved in performing an operation. For Example banking application may have:

1. Web Server to interact with end users via Browser
2. Middle Tier to validate the input and output for web server
3. Data Base to store data and procedures
4. Transaction Processor which could be a large capacity Mainframe or any other Legacy system to carry out Trillions of transactions per second.

If we talk about testing banking applications it requires an end-to-end testing methodology involving multiple software testing techniques to ensure:

- Total coverage of all banking workflows and Business Requirements
- Functional aspect of the application
- Security aspect of the application
- Data Integrity
- Concurrency
- User Experience

Typical stages involved in testing Banking Applications are shown in below workflow which we will be discussing individually.



Requirement Gathering:

Requirement gathering phase involves documentation of requirements either as Functional Specifications or Use Cases. Requirements are gathered as per customer needs and documented by Banking Experts or Business Analyst. To write requirements on more than one subject expert are involved as banking itself has multiple sub domains and one full fledged bank application will be the integration of all. For Example: A banking application may have



separate modules for Transfers, Credit Cards, Reports, Loan Accounts, Bill Payments, Tra Etc.

Requirement Review:

The deliverable of Requirement Gathering is reviewed by all the stakeholders such as QA Engineers, Development leads and Peer Business Analysts. They cross check that neither existing business workflows nor new workflows are violated.



Business Scenario Preparations:

In this stage QA Engineers derive Business Scenarios from the requirement documents (Functions Specs or Use Cases); Business Scenarios are derived in such a way that all Business Requirements are covered. Business Scenarios are high level scenarios without a detailed steps, further these Business Scenarios are reviewed by Business Analyst to ensure all of Business Requirements are met and it's easier for BAs to review high level scenarios than reviewing low level detailed Test Cases.



Test Case Preparation:

In this stage Test Cases are derived from Business Scenarios, one Business Scenario leads several positive test cases and negative test cases. Generally tools used during this stage are Microsoft Excel, Test Director or Quality Center.



Test Case Review:

Reviews by peer QA Engineers



Test Case Execution:

Test Case Execution could be either manual or automatic involving tools like QC, QTP or any other.



Database Testing:

Banking Application involves complex transactions which are performed both at UI level and Database level, therefore Database testing is as important as functional testing. Database itself is an entirely separate layer hence it is carried out by database specialists and techniques like



Data loading

Database Migration

Testing DB Schema and Data types

Rules Testing

Testing Stored Procedures and Functions

Testing Triggers

Data Integrity

Security Testing:

Security Testing is usually the last stage in the testing cycle as completing functional testing are entry criteria to commence Security testing. Security testing is one of the stages in the entire Application testing cycle as this stage ensures that application complies with Federal and Industry standards. Security testing cycle makes sure the application does not have any web vulnerability which may expose sensitive data to an intruder or an attacker and complies with standards like OWASP.



In this stage the major task involves in the whole application scan which is carried out using tools like IBM Appscan or HP WebInspect (2 Most popular tools).

Once the Scan is complete the Scan Report is published out of which False Positives are filtered out and rest of the vulnerabilities are reported to Development team for fixing

depending on the Severity.

Other Manual tools for Security Testing used are: Paros Proxy, Http Watch, Burp Suite, Fortify tools Etc.

Apart from the above stages there might be different stages involved like Integration Te and Performance Testing.

In today's scenario majority of Banking Projects are using: Agile/Scrum, RUP and Continuous Integration methodologies, and Tools packages like Microsoft's VSTS and Rational Tools. As we mentioned RUP above, RUP stands for Rational Unified Process, which is an iterative software development methodology introduced by IBM which comprises of four phases in which development and testing activities are carried out.

Inception

Four phases are:

- i)
- ii)Collaboration
- iii)Construction and
- iii) Transition

RUP widely involves IBM Rational tools.

In this article we discussed how complex a Banking application could be and what are the typical phases involved in testing the application. Apart from that we also discussed cu trends followed by IT industries including software development methodologies and tools.

Test cases for opening bank account:

Input parameters checking:

- Name -Date of Birth - Photo -Address Proof -Identity proof -Introducers (if applicable



Type of account:

- Savings account -Salary account -Joint account - Current account - Secondary account -



Test cases:

-Checking mandatory input parameters -Checking optional input parameters -Check whether the balance) -Check whether you are able to withdraw an amount in the newly created acco deposit) (and thus updating the balance) -Check whether company name and its pan number account -Check whether primary account number is provided in case of secondary account -



Result:

Thus, the Test cases for Banking Application is implemented and output is verified successfully.

Thus, the ATM system specifications and reporting the various bugs is implemented and output was verified successfully

The Library Management System is an online application for assisting a librarian in managing a book library in a university. The system would provide basic set of features to add/update clients, add/update books, search for books, and manage check-in / checkout processes. Our test group tested the system based on the requirement specification.

INTRODUCTION

This test report is the result of testing in the LMS. It mainly focuses on two problems: what we will test and how we will test.

Result

GUI test

Pass criteria: librarians could use this GUI to interface with the backend library database without any difficulties

Result: pass

Database test

Pass criteria: Results of all basic and advanced operations are normal (refer to section 4)

Result: pass

Basic function test Add a student Pass criteria:

- Each customer/student should have following attributes: Student ID/SSN (unique), Name, Address and Phone number.
Result: pass
- The retrieved customer information by viewing customer detail should contain the four attributes.
Result: pass

Update/delete student Pass criteria:

- The record would be selected using the student ID
Result: pass
- Updates can be made on full. Items only: Name, Address, Phone number
Result: pass
- The record can be deleted if there are no books issued by user.
Result: Partially pass. When no books issued by user, he can be deleted. But when there are books Issued by this user, he was also deleted. It is wrong.
- The updated values would be reflected if the same customer's ID/SSN is called for.
Result: pass
- If customer were deleted, it would not appear in further search queries.
Result: pass

Add a book

Pass criteria:

- Each book shall have following attributes: Call Number, ISBN, Title, Author name.
Result: pass
- The retrieved book information should contain the four attributes.
Result: pass

Update/delete book

Pass criteria:

- The book item can be retrieved using the call number
Result: did not pass. Cannot retrieve using the call number
- The data items which can be updated are: ISBN, Title, Author name Result: pass
- The book can be deleted only if no user has issued it.
Result: partially pass. When no user has issued it, pass. When there are user having issued it, did not pass
- The updated values would be reflected if the same call number is called for Result: pass
- If book were deleted, it would not appear in further search queries. Result: pass

Search for book

Pass criteria:

- The product shall let Librarian query books,, detail information by their ISBN number or Author or Title.
Result: pass
- The search results would produce a list of books, which match the search parameters with following Details: Call number, ISBN number, Title, Author
Result: pass
- The display would also provide the number of copies which is available for issue
Result: pass
- The display shall provide a means to select one or more rows to a user-list Result: pass
- A detailed view of each book should provide information about check-in/check out status, with the borrower's information.
Result: pass
- The search display will be restricted to 20 results per page and there would be means to navigate from sets of search results.
Result: pass
- The user can perform multiple searches before finally selecting a set of books for check in or checkout. These should be stored across searches.
Result: pass
- A book may have more than one copy. But every copy with the same ISBN number should have same detail information.
Result: pass
- The borrower's list should agree with the data in student's account Result: pass

Check-in book

Pass criteria:

- ☐ Librarians can check in a book using its call number
Result: pass
- ☐ The check-in can be initiated from a previous search operation where user has selected a set of books.
Result: pass
- ☐ The return date would automatically reflect the current system date. Result: did not pass.
- ☐ Any late fees would be computed as difference between due date and return date at rate of 10 cents a day.
Result: did not pass
- ☐ A book, which has been checked in once, should not be checked in again Result: pass

Check-out book

Pass criteria:

- ☐ Librarians can check out a book using its call number Result: pass
- ☐ The checkout can be initiated from a previous search operation where user has selected a set of books.
Result: pass
- ☐ The student ID who is issuing the book would be entered Result: pass
- ☐ The issue date would automatically reflect the current system date. Result: did not pass
- ☐ The due date would automatically be stamped as 5 days from current date. Result: did not pass

- ☐ A book, which has been checked out once, should not be checked out again
Result: pass

- ☐ A student who has books due should not be allowed to check out any books
Result: did not pass

- ☐ The max. No of books that can be issued to a customer would be 10. The system should not allow checkout of books beyond this limit.
Result: pass View book detail Pass criteria:

- ☐ This view would display details about a selected book from search operation
Result: pass
- ☐ The details to be displayed are: Call number, IBN, Title, Author, Issue status (In library or checked out), If book is checked out it would display, User ID & Name, Checkout date, Due date
Result: for checkout date and due date, did not pass
- ☐ Books checked in should not display user summary
Result: pass
- ☐ Books checked out should display correct user details. Result: pass

View student detail Pass criteria:

- ☐ Librarians can select a user record for detailed view Result: pass
- ☐ The detail view should show:
 - a. User name, ID, Address & Phone number Result: pass
 - b. The books issued by user with issue date, due date, call number, title Result: did not pass
 - c. Late fees & Fines summary and total Result: did not pass
- ☐ The display should match existing user profile Result: pass
- ☐ The books checked out should have their statuses marked Result: pass
- ☐ The book search query should show the user id correctly. Result: pass

Network test

Pass criteria: Results of operations (ping, ftp and ODBC connectivity check) are normal
Result: did not test this item, because no enough machines and no available environment.

Result:

Thus, the Test cases for library application is implemented and output is verified successfully.

Aim:

To install Pytest and to write test cases for a program and to test using Pytest.

Pytest:

It is a testing framework that allows users to write test codes using python programming language. It helps you to write simple and scalable test case for databases, API's or UI. Pytest is mainly used for writing tests from simple unit tests to complex functional tests.

Procedure:

Installation on Windows:

- 1) Navigate to folder where the python is installed.
- 2) Open scripts folder and copy the address location.
- 3) Open command prompt and execute "cd copied_address_Location".
- 4) Execute "pip install pytest".

Creating File:

- 1) The file to be tested should have name as test_*.py or *_test.py
- 2) File should be located in the same folder where pytest module is installed

Creating a sample program:

- 1) Function to be tested should has name def test_*();
- 2) Inside the function the line to be tested should has assert keyword at the beginning.

Terminologies in Pytest:

F – Failed
.
– Passed S
– Skipped X
– xpassed x
– xfailed

Eg:

Parameterized Addition Program:

File name: test_parameter.py

```
import pytest
@pytest.mark.parametrize("input1, input2, output",[(5,5,10),(3,5,12)])

def test_add(input1, input2, output):
    assert input1 + input2 == output
```

for execution : execute "pytest test_parameter.py".

Output:

```
E:\Python\Scripts>pytest test_parameter.py
===== test session starts =====
platform win32 -- Python 3.10.5, pytest-7.1.2, pluggy-1.0.0
rootdir: E:\Python\Scripts
collected 2 items

test_parameter.py .F [100%]

===== FAILURES =====
_____ test_add[3-5-12] _____

input1 = 3, input2 = 5, output = 12

    @pytest.mark.parametrize("input1,input2,output",[(5,5,10),(3,5,12)])
    def test_add(input1,input2,output):
>         assert input1+input2 == output
E         assert (3 + 5) == 12

test_parameter.py:5: AssertionError
===== short test summary info =====
FAILED test_parameter.py::test_add[3-5-12] - assert (3 + 5) == 12
===== 1 failed, 1 passed in 0.05s =====
```

Result:

Thus, we have installed pytest, implemented and executed a parameterized addition program and the output is verified successfully.

Ex.No: 12 Pytest Python program for Addition

DATE: 22/10/2024

REGISTER NUMBER : 212222040036

AIM:

To write a python program for addition of two numbers and test the test cases using Pytest.

Algorithm:

1. Write the python program for addition of two numbers
2. Make sure that function name should be "def test_*():" and the line to be tested should have assert keyword at the beginning
3. Write some test cases for to be tested and save it as "test_add.py".
4. Open command prompt and change the directory to where pytest and program is saved and type "pytest test_add.py" and run it.

Program:

```
def add(a,b):  
    return a+b  
  
def test_3_plus_5_equals_8():  
    assert add(3,5) == 8  
  
def test_2_plus_3_equals_5():  
    assert add(2,3) == 6
```



Output:

```
E:\Python\Scripts>pytest test_add.py  
===== test session starts =====  
platform win32 -- Python 3.10.5, pytest-7.1.2, pluggy-1.0.0  
rootdir: E:\Python\Scripts  
collected 2 items  
  
test_add.py .F [100%]  
  
===== FAILURES =====  
test_2_plus_3_equals_5  
  
def test_2_plus_3_equals_5():  
>     assert add(2,3) == 6  
E       assert 5 == 6  
E       + where 5 = add(2, 3)  
  
test_add.py:20: AssertionError  
===== short test summary info =====  
FAILED test_add.py::test_2_plus_3_equals_5 - assert 5 == 6  
===== 1 failed, 1 passed in 0.05s =====
```

Result:

Thus, the python program for addition is tested using pytest and executed and output is verified successfully.

Ex.No: 13 Pytest program for Sum of Digits

DATE: 29.10.2024

REGISTER NUMBER : 212222040036

AIM:

To write a python program for sum of digits and validate the code using Pytest.

Algorithm:

1. Write the python program for sum of digits of a number.
2. Make sure that function name should be "def test_*():" and the line to be tested should have assert keyword at the beginning.
3. Write some test cases for to be tested and save it as "test_sumofdig.py".
4. Open command prompt and change the directory to where pytest is installed
5. Executethe program as "pytest test_sumofdig.py".
6. Stop the program.

Program:

```
def sumOfDigits(n):
    sum = 0
    while (n != 0):
        sum = sum + int(n % 10)
        n = int(n/10)
    return sum
def test_1():
    assert sumOfDigits(123) == 6
def test_2():
    assert sumOfDigits(256) == 2
```



Output:

```
E:\Python\Scripts>pytest test_sumofdig.py
===== test session starts =====
platform win32 -- Python 3.10.5, pytest-7.1.2, pluggy-1.0.0
rootdir: E:\Python\Scripts
collected 2 items

test_sumofdig.py .F [100%]

===== FAILURES =====
test_2

  def test_2():
>     assert sumOfDigits(256) == 2
E       assert 13 == 2
E       + where 13 = sumOfDigits(256)

test_sumofdig.py:12: AssertionError
===== short test summary info =====
FAILED test_sumofdig.py::test_2 - assert 13 == 2
===== 1 failed, 1 passed in 0.08s =====
```

Result:

Thus, the python program for sum of digits is tested using pytest and executed and output is verified successfully.

Ex.No: 14 Pytest Python program for Fibonacci Series

DATE: 05/11/2024

REGISTER NUMBER : 212222040036

AIM:

To write a python program for Fibonacci Series and test the test cases using Pytest.

Algorithm:

1. Write the python program for addition of two numbers
2. Make sure that function name should be "def test_*():" and the line to be tested should have assert keyword at the beginning
3. Write some test cases for to be tested and save it as "test_add.py".
4. Open command prompt and change the directory to where pytest and program is saved and type "pytest test_add.py" and run it.

Program:

```
def fibR(n):
    if n==1 or n==2:
        return 1
    return fibR(n-1)+fibR(n-2)
def test_fib_1_equals_1():
    assert fibR(1) == 1
def test_fib_2_equals_1():
    assert fibR(2) == 1
def test_fib_6_equals_8():
    assert fibR(6) == 7
```



Output:

```
E:\Python\Scripts>pytest test_fib.py
===== test session starts =====
platform win32 -- Python 3.10.5, pytest-7.1.2, pluggy-1.0.0
rootdir: E:\Python\Scripts
collected 3 items

test_fib.py ..F [100%]

===== FAILURES =====
test_fib_6_equals_8

def test_fib_6_equals_8():
>     assert fibR(6) == 7
E       assert 8 == 7
E       + where 8 = fibR(6)

test_fib.py:13: AssertionError
===== short test summary info =====
FAILED test_fib.py::test_fib_6_equals_8 - assert 8 == 7
===== 1 failed, 2 passed in 0.11s =====
```

Result:

Thus, the python program for addition is tested using pytest and executed and output is verified successfully.

What is Selenium?

JavaScript framework that runs in your web browser Works anywhere JavaScript is supported Hooks for many other languages Java, Ruby, Python Can simulate a user navigating through pages and then assert for specific marks on the pages All you need to really know is HTML to start using it right away

Selenium IDE

Selenium Integrated Development Environment (IDE) is a Firefox plugin that lets testers record their actions as they follow the workflow that they need to test.

It provides a Graphical User Interface for recording user actions using Firefox which is used to learn and use Selenium, but it can only be used with Firefox browser as other browsers are not supported.

However, the recorded scripts can be converted into various programming languages supported by Selenium and the scripts can be executed on other browsers as well.

Selenium-IDE Download

Step 1 – Launch Firefox and navigate to the following URL - <http://seleniumhq.org/download/>.

Under the Selenium IDE section, click on the link that shows the current version number as shown below.



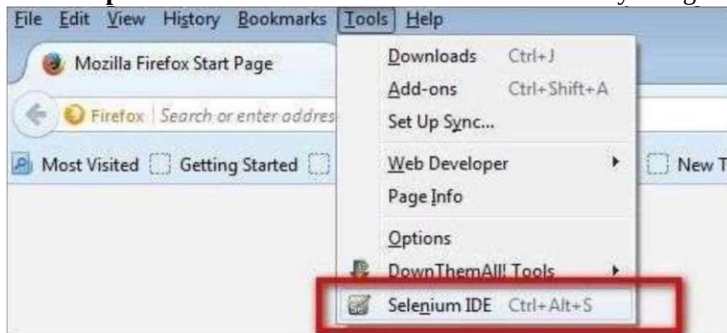
Step 2 – Firefox add-ons notifier pops up with allow and disallow options. User has to allow the installation.



Step 3 – The add-ons installer warns the user about untrusted add-ons. Click 'Install Now'.



Step 4 – The Selenium IDE can now be accessed by navigating to Tools >>Selenium IDE.



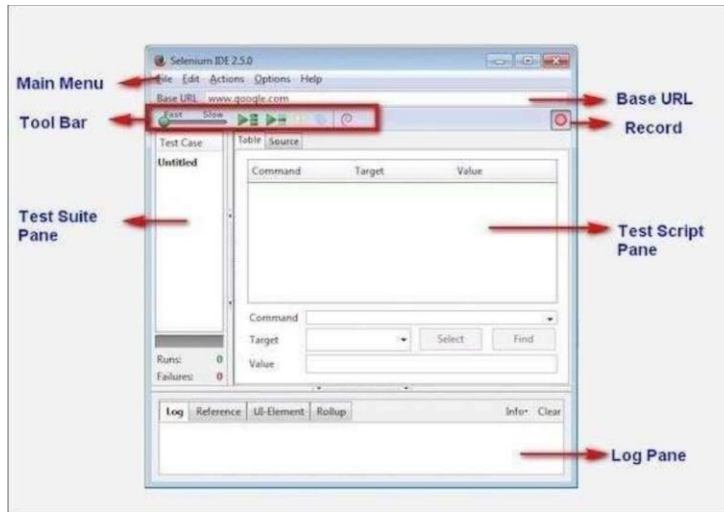
Step 5 – The Selenium IDE can also be accessed directly from the quick access menu bar as shown below.



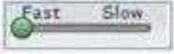
Selenium IDE Features

This section deals with the features available in Selenium IDE.

The following image shows the features of Selenium IDE with the help of a simple tool-tip.



The features of the record tool bar are explained below.

Control	Control Name	Description
	Speed Control	This helps in controlling the speed of the test case runs.
	Run All	Executes the entire test suite that contains multiple test cases.
	Run	Executes the currently selected test.
	Pause/Resume	Allows user to pause or resume the script execution. Enabled only during the execution.
	Step	Helps user to debug the test by executing only one step of a test case at a time.
	Test Runner Mode	Allows user to execute the test case in a browser loaded with the selenium Core. It is an obsolete functionality that likely to be deprecated.
	Apply Rollup Rules	This features allows repetitive sequences of selenium commands to be grouped into a single action.
	Record	This features helps user to Records the user's browser actions.

Creating Selenium IDE Tests

This section deals with how to create IDE tests using recording feature. The following steps are involved in

creating Selenium tests using IDE –

- Recording and adding commands in a test
- Saving the recorded test
- Saving the test suite
- Executing the recorded test

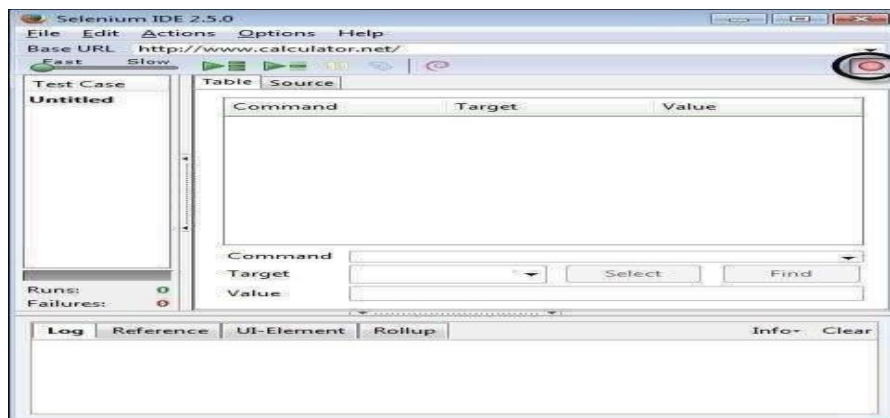
Recording and Adding

Commands in a Test

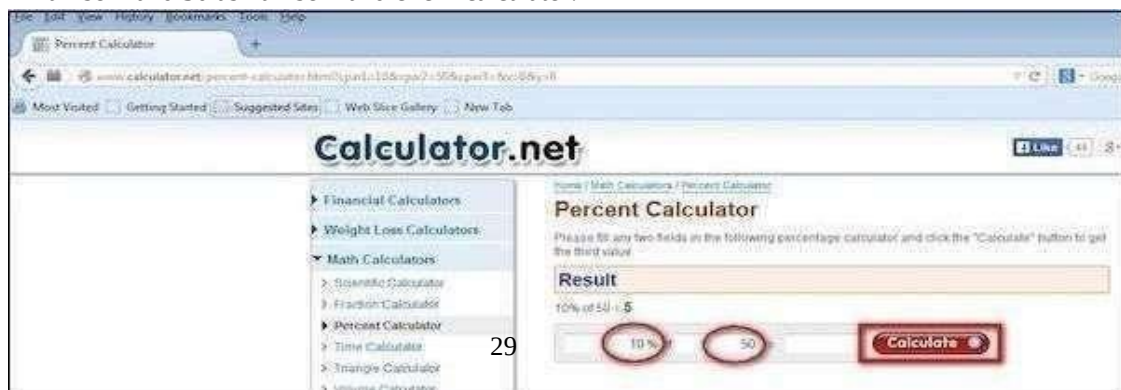
We will use www.ncalculators.com to demonstrate the features of Selenium.

Step 1 – Launch the Firefox browser and navigate to the website - <https://www.ncalculators.com/>

Step 2 – Open Selenium IDE from the Tools menu and press the record button that is on the top- right corner.



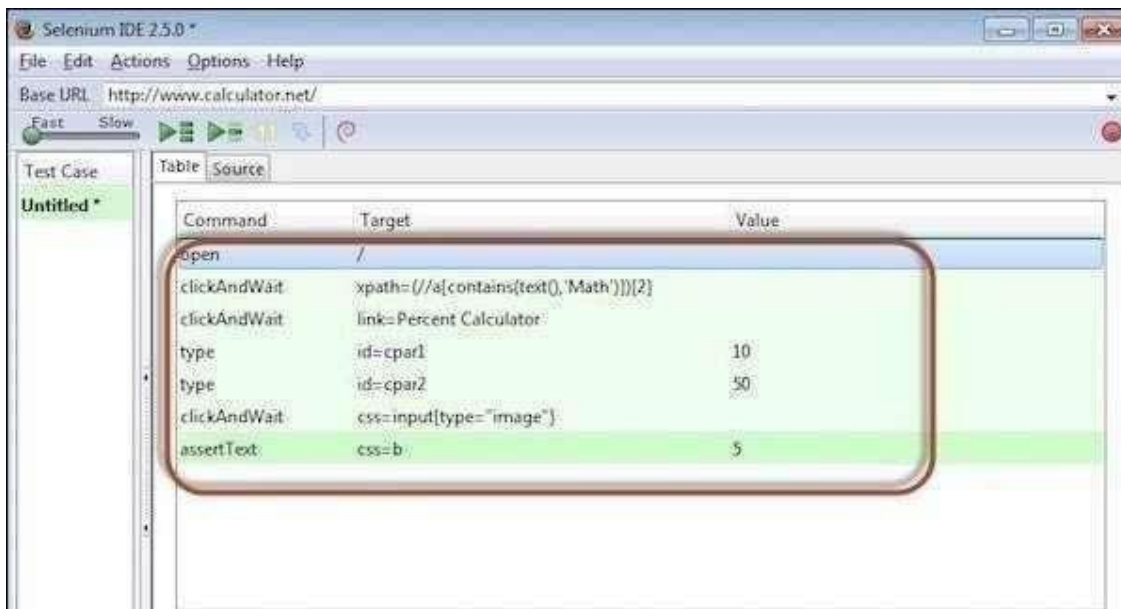
Step 3 – Navigate to "Math Calculator" >> "Percent Calculator" >> enter "10" as number1 and 50 as number2 and click "calculate".



Step 4 – User can then insert a checkpoint by right clicking on the webelement and select "Show all available commands" >> select "assert text css=b 5"



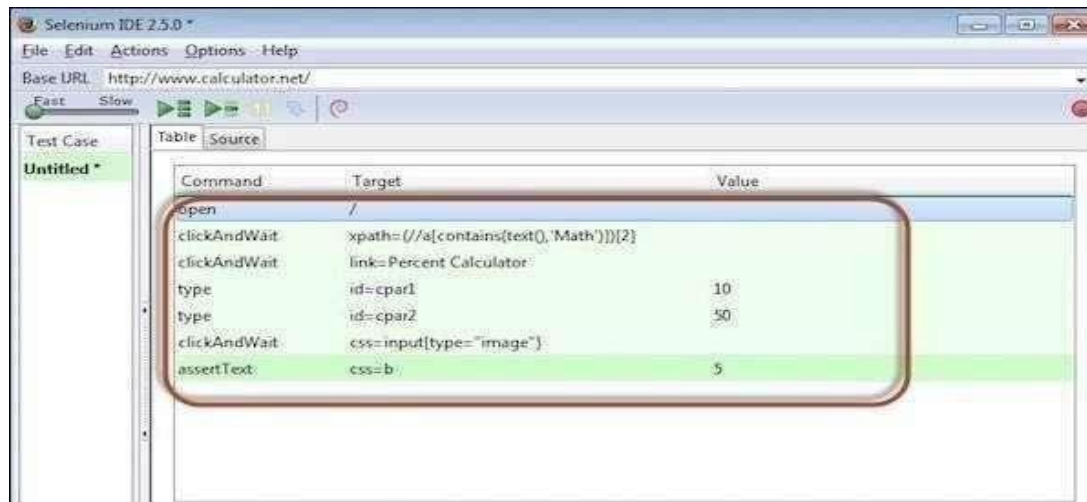
Step 5 – The recorded script is generated and the script is displayed as shown below.



Saving the Recorded Test

Step 1 – Save the Test Case by navigating to "File" >> "Save Test" and save the file in the location of your choice. The file is saved as .HTML as default.

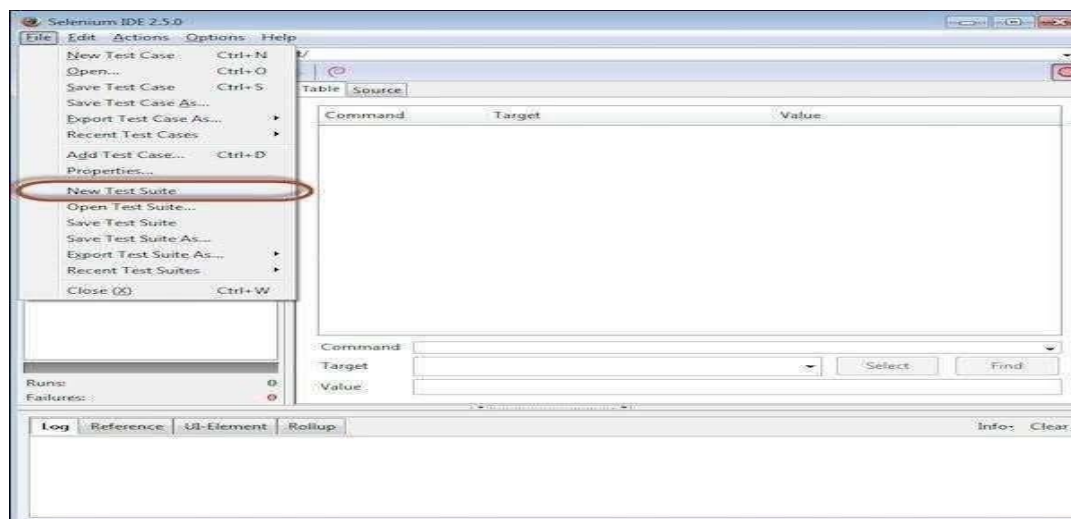
The test can also be saved with an extension htm, shtml, and xhtml.



Saving the Test Suite

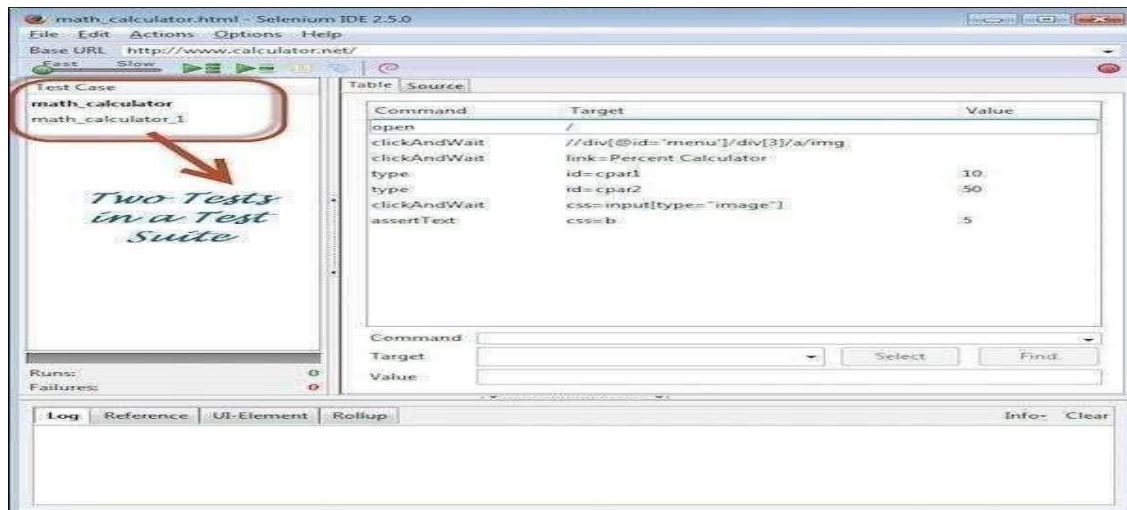
A test suite is a collection of tests that can be executed as a single entity.

Step 1 – Create a test suite by navigating to "File" >> "New Test Suite" as shown below.



Step 2 – The tests can be recorded one by one by choosing the option "New TestCase" from the "File" Menu.

Step 3 – The individual tests are saved with a name along with saving a "Test Suite".



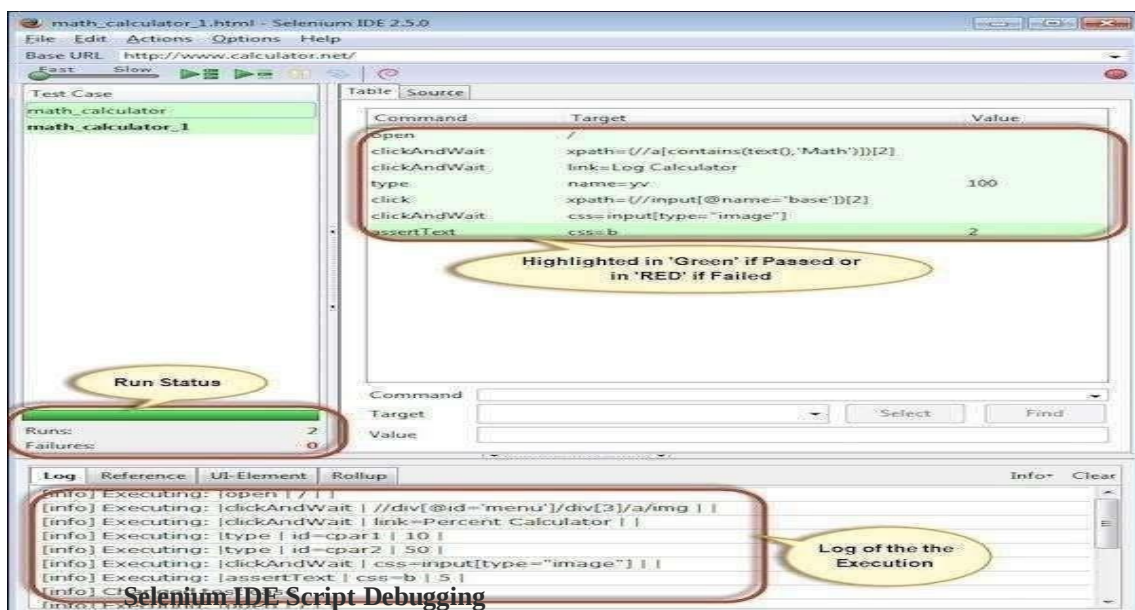
Executing the Recorded Test

The recorded scripts can then be executed either by clicking "Play entire suite" or "Play current test" button in the toolbar.

Step 1 – The Run status can be seen in the status pane that displays the number of tests passed and failed.

Step 2 – Once a step is executed, the user can see the result in the "Log" Pane.

Step 3 – After executing each step, the background of the test step turns "Green" if passed and "Red" if failed as shown below.



Selenium IDE Script Debugging

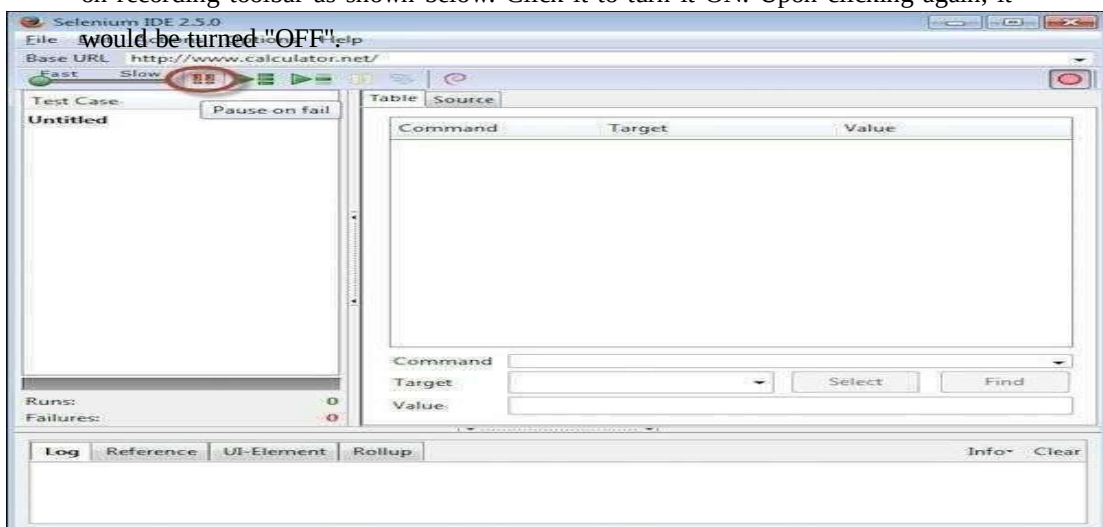
This section deals with debugging the Selenium IDE script.

Debugging is the process of finding and fixing errors in the test script. It is a common step in any script development. To make the process more robust, we can make use a plugin "Power Debugger" for Selenium IDE.

Step 1 – To install Power Debugger for Selenium IDE, navigate to <https://addons.mozilla.org/en-US/firefox/addon/power-debugger-selenium-ide/> and click "Add to Firefox" as shown below.



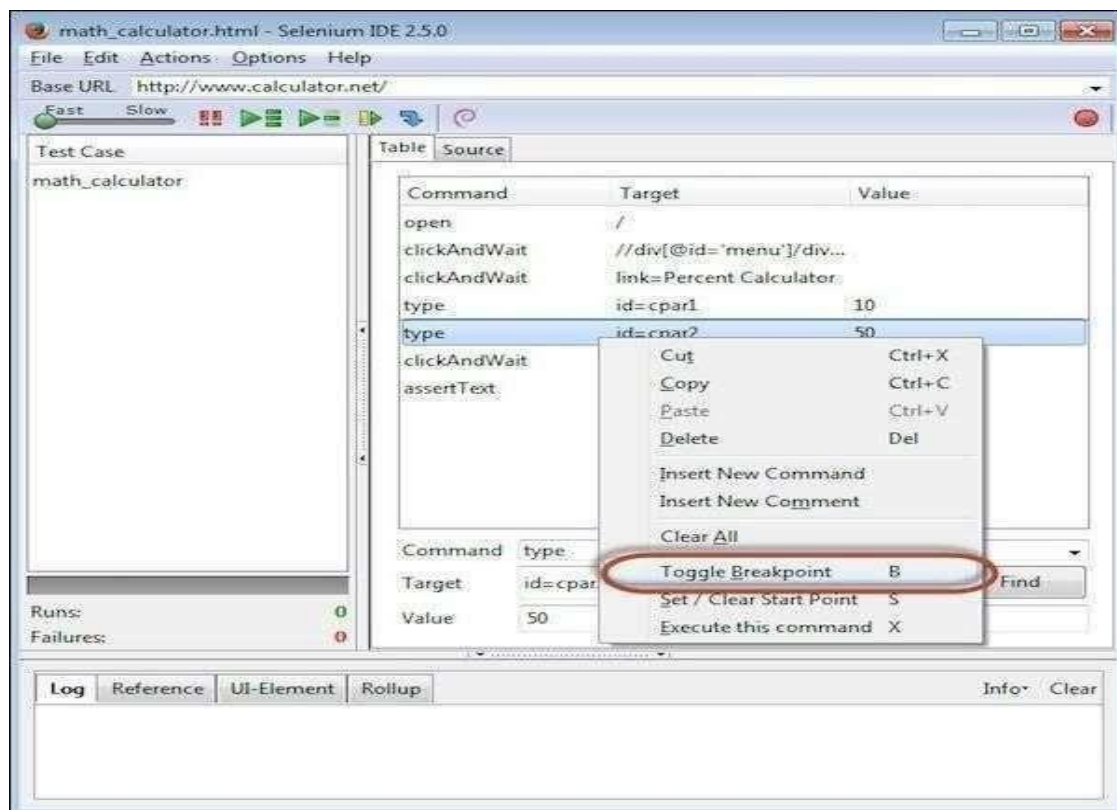
Step 2 – Now launch 'Selenium IDE' and you will notice a new icon, "Pause on Fail" on recording toolbar as shown below. Click it to turn it ON. Upon clicking again, it would be turned "OFF".



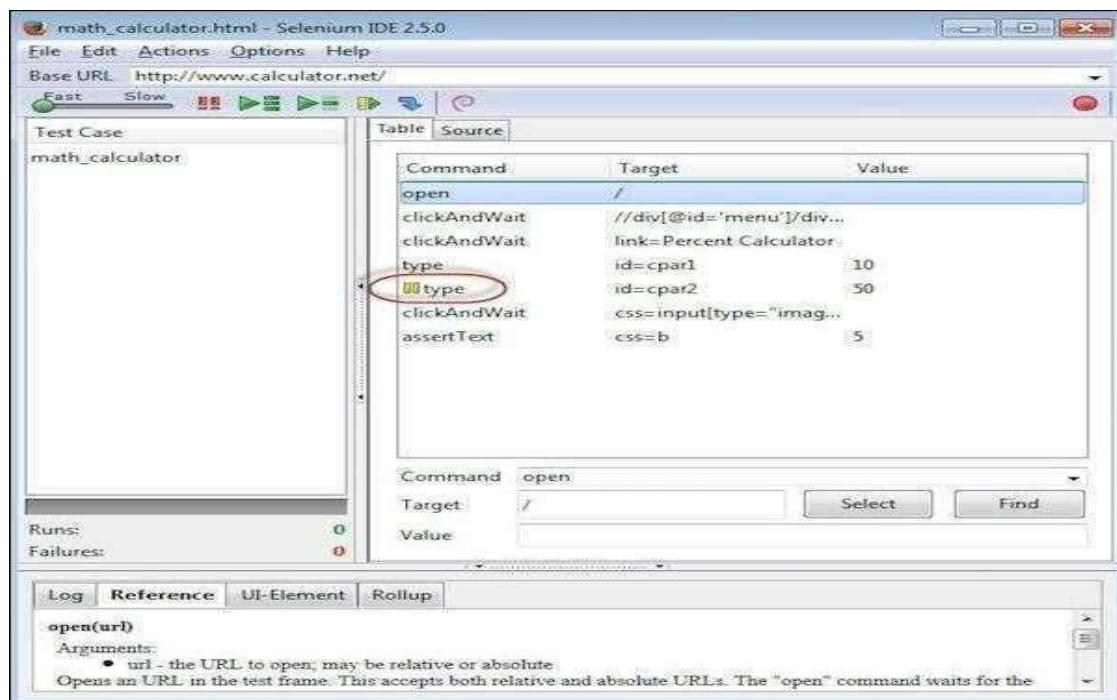
Step 3 – Users can turn "pause on fail" on or off any time even when the test is running.

Step 4 – Once the test case pauses due to a failed step, you can use the resume/step buttons to continue the test execution. The execution will **NOT** be paused if the failure is on the last command of any test case.

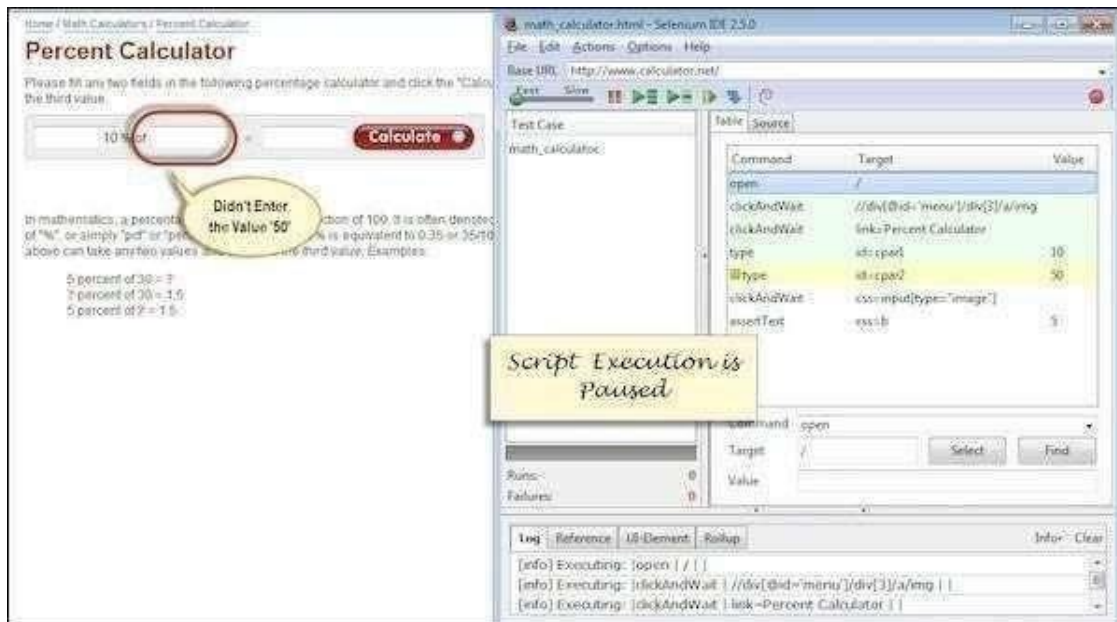
Step 5 – We can also use breakpoints to understand what exactly happens during the step. To insert a breakpoint on a particular step, "Right Click" and select "Toggle Breakpoint" from the context- sensitive menu.



Step 6 – Upon inserting the breakpoint, the particular step is displayed with a pause icon as shown below.



Step 7 – When we execute the script, the script execution is paused where the breakpoint is inserted. This will help the user to evaluate the value/presence of an element when the execution is in progress.

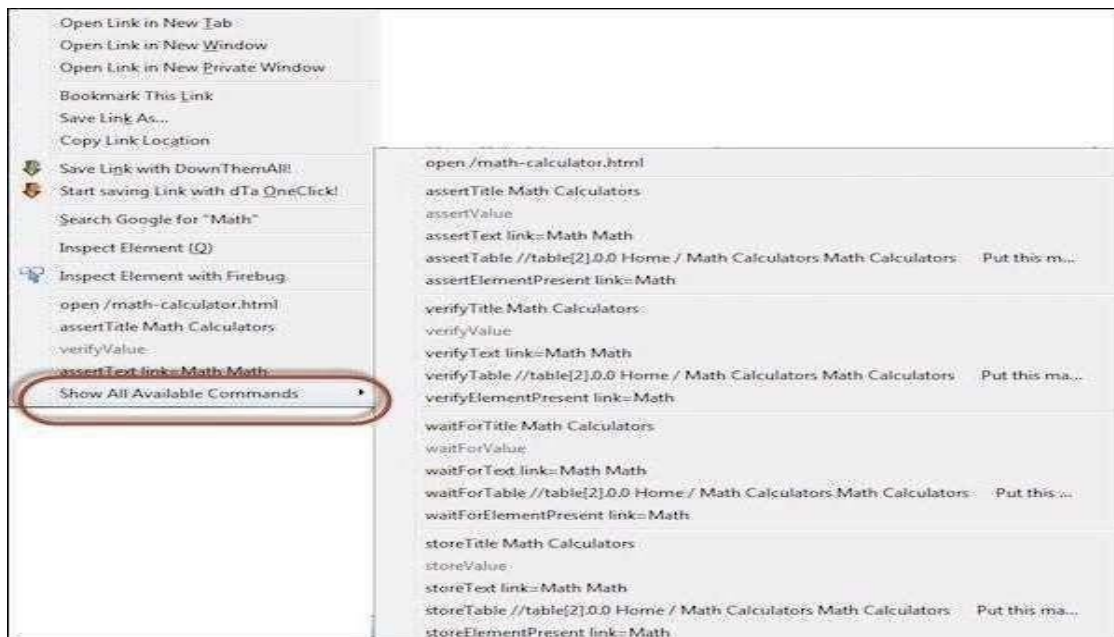


Inserting Verification Points

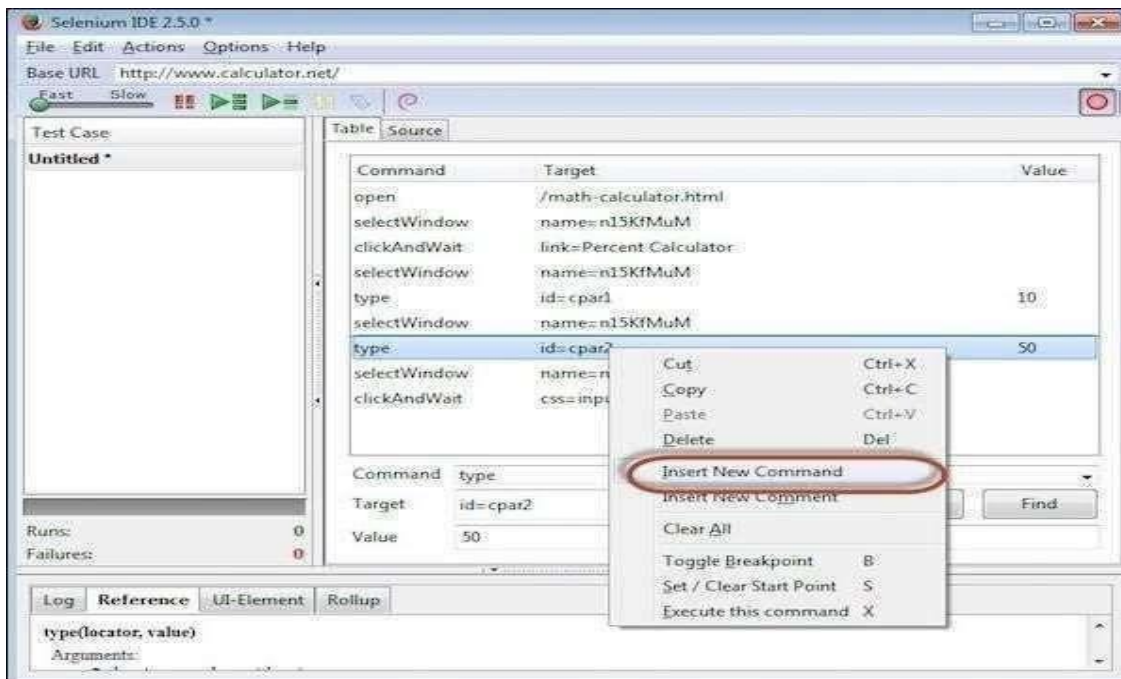
This section describes how to insert verification points in Selenium IDE.

The test cases that we develop also need to check the properties of a web page. It requires assert and verify commands. There are two ways to insert verification points into the script.

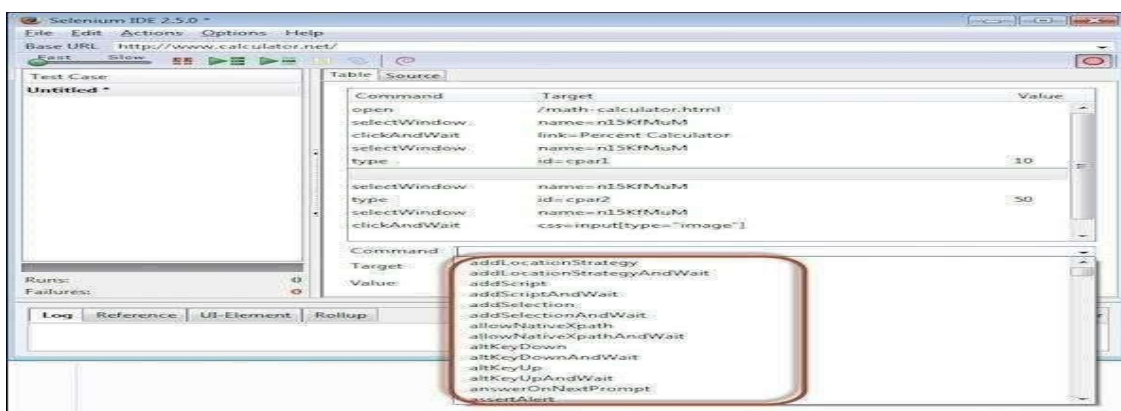
To insert a verification point in recording mode, "Right click" on the element and choose "Show all Available Commands" as shown below.



We can also insert a command by performing a "Right-Click" and choosing "Insert New Command".



After inserting a new command, click 'Command' dropdown and select appropriate verification point from the available list of commands as shown below.



Given below are the mostly used verification commands that help us check if a particular step has passed or failed.

- verifyElementPresent
- assertElementPresent
- verifyElementNotPresent
- assertElementNotPresent
- verifyText
- assertText
- verifyAttribute
- assertAttribute
- verifyChecked
- assertChecked

- verifyAlert
- assertAlert
- verifyTitle
- assertTitle

SynchronizationPoints

During script execution, the application might respond based on server load, hence it is required for

the application and script to be in sync. Given below are few a commands that we can use to ensure that the script and application are in sync.

- waitForAlertNotPresent
- waitForAlertPresent
- waitForElementPresent
- waitForElementNotPresent
- waitForTextPresent
- waitForTextNotPresent
- waitForPageToLoad
- waitForFrameToLoad

Selenium Pattern Matching

- This section deals with how to work with regular expressions using IDE.

Like locators, patterns are a type of parameter frequently used by Selenium. It allows users to describe patterns with the help of special characters. Many a time, the text that we would like to verify are dynamic; in that case, pattern matching is very useful.

Pattern matching is used with all the verification point commands - verifyTextPresent, verifyTitle, verifyAlert, assertConfirmation, verifyText, and verifyPrompt.

There are three ways to define a pattern –

- Globbing
- regular expressions, and
- exact patterns.

Globbing

Most techies who have used file matching patterns in Linux or Windows while searching for a certain file type like *.doc or *.jpg. would be familiar with term "globbing"
Globbing in Selenium supports only three special characters: *, ?, and [].

- * – matches any number of characters.
- ? – matches a single character.

□ [] – called a character class, lets you match any single character found within the brackets. [0- 9] matches any digit.

To specify a glob in a Selenium command, prefix the pattern with the keyword 'glob:'. For example, if you would like to search for the texts "tax year 2013" or "tax year 2014", then you can use the glob "tax year *" as shown below.

However the usage of "glob:" is optional while specifying a text pattern because globbing patterns are the default in Selenium.

Command	Target	Value
clickAndWait	link = search	
verifyTextPresent	glob: tax year *	

ExactPatterns

Patterns with the prefix 'exact:' will match the given text as it is. Let us say, the user wants an exact match with thevalue string, i.e., without the glob operator doing its work, one can use the 'exact' pattern as shown below. In this example the operator '*' will work as a normal character rather than a pattern- matching wildcard character.

Command	Target	Value
clickAndWait	link = search	
verifyValue	exact: *.doc	

RegularExpressions

Regular expressions are the most useful among the pattern matching techniques available. Selenium supports the complete set of regular expression patterns that Javascript supports. Hence the users are no longer limited by *, ? and [] globbing patterns.

To use RegEx patterns, we need to prefix with either "regexp:" or "regexpi:". The prefix "regexpi" is case- insensitive. The glob: and the exact: patterns are the subsets of the Regular Expression patterns. Everything that is done with glob: or exact: can be accomplished with the help of RegExp.

Example

For example, the following will test if an input field with the id 'name' contains the string 'tax year', 'Tax Year', or 'tax Year'.

Command	Target	Value
clickAndWait	link = search	
verifyValue	id = name	regexp:[Tt]ax ([Yy]ear)

Selenium User Extensions

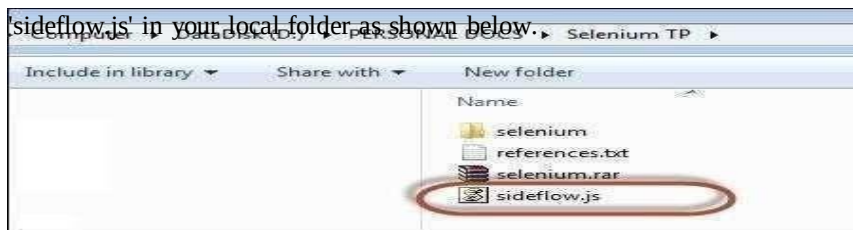
The Java script that allows users to customize or add new functionality.

It is easy to extend Selenium IDE by adding customized actions, assertions, and locator-strategies. It is done with the help of JavaScript by adding methods to the Selenium object prototype. On startup, Selenium will automatically look through the methods on these prototypes, using name patterns to recognize which ones are actions, assertions, and locators.

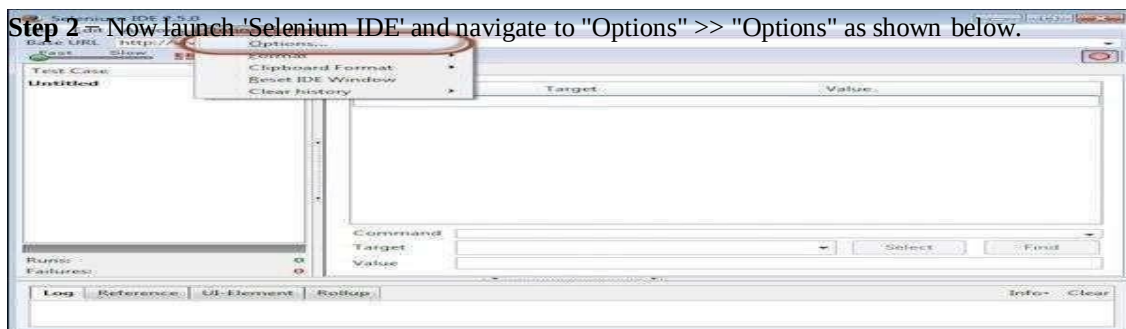
Let us add a 'while' Loop in Selenium IDE with the help of JavaScript.

Step 1 – To add the js file, first navigate

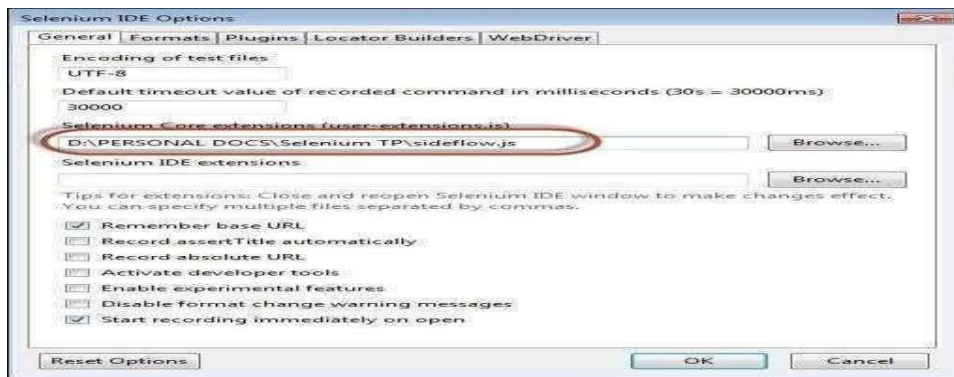
to <https://github.com/darrenderidder/sideflow/blob/master/sideflow.js> and copy the script and place save it as 'sideflow.js' in your local folder as shown below.



Step 2 Now launch 'Selenium IDE' and navigate to "Options" >> "Options" as shown below.



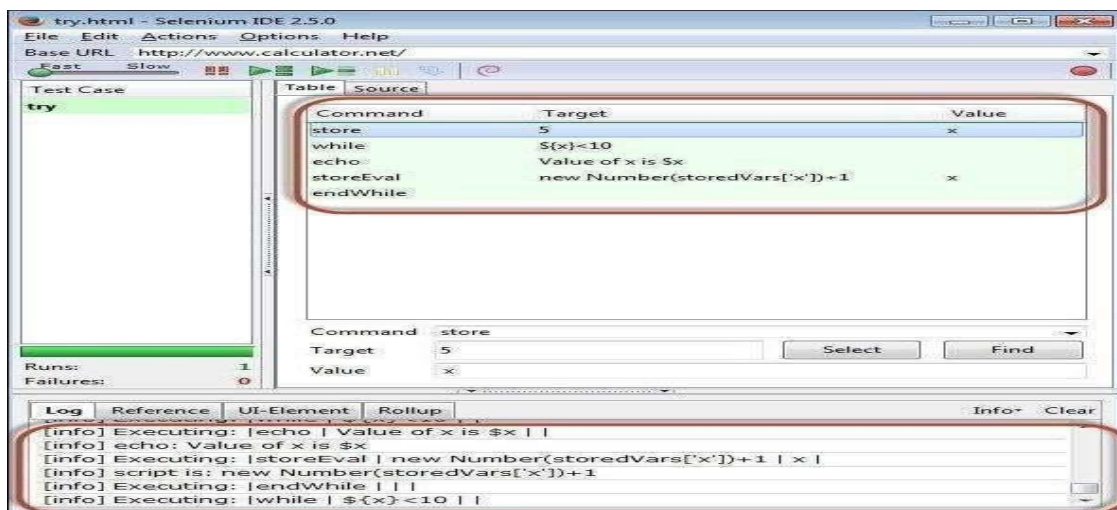
Step 3 – Click the 'Browse' button under 'Selenium Core Extensions' area and point to the js file that we have saved in Step 1.



Step 4 – Restart Selenium IDE.

Step 5 – Now you will have access to a few more commands such as "Label", "While" etc.

Step 6 – Now we will be able to create a While loop within Selenium IDE and it will execute as shown below.



Result: Thus, the study of selenium web testing tool is conducted and the results were noted.